Informe final – Sistema de Movilidad Compartida

Programación Orientada a Objetos · Propuesta C · Etapa 2 Universidad Argentina de la Empresa · Docente: María Ángela León

Integrantes: Astudillo Solis, Valentina (1211931) · Pacheco, Maitena Alumine (1216103) · Romero Chacon, Marcos Ignacio (1219650)

Repositorio: https://github.com/Mromeroch/POO_TPO.git

1. Introducción

Este informe cierra el trabajo práctico. Resume qué se implementó en la Etapa 2, en qué se apartó el código del diseño de la Etapa 1 y por qué, qué dificultades aparecieron, cómo se repartió el trabajo y qué aprendió el grupo.

El sistema es una aplicación de consola en Java 17 para un servicio municipal de bicicletas y monopatines compartidos. Tiene cuatro roles (usuario, operador, técnico y administrador), persistencia en archivos CSV y pruebas unitarias con JUnit 5. Las instrucciones de ejecución están en el README.

2. Estado final de la implementación

Casos de uso	Estado	Observaciones
CU-01 a CU-04 (registro, cuenta y saldo, medios de pago, historial)	Implementados	Menú de usuario.
CU-05 a CU-13 (retiro, devolución, tarifa y promociones)	Implementados	Los dos casos modelados en secuencia (CU-05 y CU-08) siguen el orden de validaciones del diagrama, con sus ramas alt y opt.

CU-14 (reportar falla)	Implementado	Disponible para usuario y operador.
CU-15 (gestionar vehículos)	Parcial	El operador ve los vehículos de cada estación con su estado y puede reportar fallas y generar órdenes. No hay un listado general de todos los vehículos con sus reportes.
CU-16 a CU-20 (órdenes de mantenimiento)	Implementados	Generar la orden y asignar el técnico son dos pasos separados (ver sección 3.2).
CU-21 y CU-22 (redistribución y traslados)	Implementados	La operación es atómica: si algo no es válido, no se mueve ningún vehículo.
CU-23 y CU-24 (gestionar tarifas y promociones)	Parcial	La lógica existe (`ServicioTarifas.registrarTarifa` con fecha de vigencia, promociones con período de vigencia y `ControladorUsuarios.setPrecioPlan`), pero no hay opciones de menú para el administrador. Las tarifas y promociones se cargan por configuración por defecto y no se persisten.
CU-25 y CU-26 (reportes)	Implementados	Viajes por usuario con rango de fechas opcional, uso por estación y vehículos con más fallas.

Además se implementaron funciones no exigidas como caso de uso propio: nota de ajuste (RF14), detección de vehículos perdidos al iniciar y a pedido del operador (RF09), registro de puntos del recorrido y cálculo de distancia con la fórmula de Haversine (RF06).

3. Cambios respecto del diseño original

La consigna pide aclarar en qué se apartó el código del modelo y justificarlo. Los cambios se agrupan en tres tipos.

3.1 Clases y elementos que se agregaron o cambiaron en el código

Cambio	Motivo
Se agregó la interfaz `EvaluadorPromociones`, implementada por `ServicioPromociones`. Estaba mencionada en el punto 5.1.3 (DIP) del informe, pero no figuraba en el diagrama de clases final.	`ControladorViajes` depende de una abstracción y no de la clase concreta. Facilita probar y cambiar la forma de evaluar promociones.
Se agregaron `RepositorioMantenimiento` (órdenes, traslados y notas de ajuste) y `RepositorioPersonal` (empleados).	Separar el acceso a datos por responsabilidad, para que cada controlador dependa solo de lo que usa.
`RepositorioMemoria` implementa todas las interfaces de repositorio, y `RepositorioArchivos` hereda de ella. En el diagrama, `RepositorioArchivos` implementaba las interfaces directamente.	Los datos viven en memoria y se vuelcan a disco completos después de cada cambio. Así las pruebas corren sin tocar el disco y no se duplica la lógica de acceso.
Se agregó la clase `Sistema` como raíz de composición.	Es el único lugar donde se crean las clases concretas y se inyectan las dependencias (DIP). Las pruebas arman un sistema completo con un reloj de prueba.
Se agregó `UsoEstacion`, una fila del reporte de uso por estación.	Evita devolver estructuras genéricas desde `ServicioReportes`.
Se agregaron métodos `restaurar*` en `Vehiculo`, `VehiculoElectrico`, `Usuario`, `CuentaUsuario`, `PlanMensual` y `OrdenMantenimiento`.	La persistencia necesita reconstruir el estado guardado sin disparar reglas de negocio (por ejemplo, restaurar una falla no debe activar el mantenimiento obligatorio).

`Vehiculo` tiene un atributo `mantenimientoPendiente` y un método `actualizarTrasViaje` que usa `descontarBateria`, `estadoTrasViaje` y `reponerBateria`, sobrescritos por `VehiculoElectrico`.	Implementa S08 (si el vehículo está en uso, el mantenimiento obligatorio se aplica al terminar el viaje) y concentra en la jerarquía la diferencia entre bicicleta común y eléctricos (Polimorfismo). `reponerBateria()` es un método `void` sobrescrito: la bicicleta común no hace nada.
`Viaje` incorpora `instanteEstacionLlena`, `minutosFacturables()`, `calcularDistancia()` y la validación `exigirEnCurso()`.	Implementan S03 (gracia por estación llena), S13 (distancia por puntos) y RN15 (un viaje cerrado no se modifica).
`Estacion.evaluarAlerta()` devuelve un `Optional<AlertaEstacion>`.	La estación es la experta en su ocupación (S15). El controlador solo guarda la alerta si existe.
`Usuario` incorpora `getPlanActivo()`, `registrarViajeFinalizado()` y `reevaluarHabilitacion()`.	S05, S06 y S17: el usuario conoce su plan y su historial, y se rehabilita cuando su saldo vuelve a ser mayor o igual a cero.
Excepciones propias específicas (`SaldoInsuficienteException`, `UsuarioInhabilitadoException`, `ViajeAbiertoException`, `VehiculoNoDisponibleException`, `EstacionLlenaException`, `RetiroNoCancelableException`, `OrdenIncompletaException`, `TarifaNoVigenteException`, entre otras) con raíz abstracta `MovilidadException`.	Cada rama de error de los diagramas de secuencia tiene su excepción. La raíz es verificada, así que los controladores la declaran con `throws` y la interfaz la captura.

El `Viaje` conserva una copia de su tarifa (tipo, cargo, precio y vigencia) en el archivo `viajes.csv`.	S16: el viaje debe conservar la tarifa con la que se calculó, aunque después cambien las tarifas vigentes.

3.2 Cambios en el comportamiento respecto de los casos de uso y secuencias

- **CU-16 y CU-17:** en el informe, Asignar técnico es un `include` de Generar orden. En el código son dos operaciones consecutivas del operador (`generarOrden` y luego `asignarTecnico`). Se separaron porque una orden puede quedar generada y pendiente de asignación, tal como contempla la alternativa A1 de CU-17.
- **Cancelación de retiro:** si la estación de retiro se llenó mientras el usuario viajaba, no hay lugar para reponer el vehículo. Se agregó este rechazo (`OperacionNoPermitidaException`), que no estaba contemplado en el diseño.
- **Cierre de orden de mantenimiento:** el técnico indica la estación de destino y la orden no se cierra si esa estación está llena (S18).
- **Devolución:** `devolverVehiculo` tiene dos versiones, con y sin distancia informada, para cubrir los viajes que no registraron puntos de recorrido (S13).
- **Retiro:** las validaciones se ejecutan en el orden del diagrama de secuencia 4.1 y cada rama lanza su excepción.

3.3 Cómo se ven los patrones en el código

La consigna exige que los principios y patrones elegidos se vean en el código y no solo en el informe.

Principio o patrón	Dónde se ve
SRP	`ControladorViajes` coordina; `ServicioTarifas`, `ServicioPromociones`, `CuentaUsuario` y `Viaje` tienen cada uno su responsabilidad. Los controladores de usuarios, mantenimiento, operaciones y reportes también están separados.
OCP	`PoliticaTarifa` y `Promocion` son abstracciones. Una tarifa o promoción nueva es una subclase nueva, sin modificar los servicios.

DIP	Los controladores dependen de las interfaces de repositorio, de `EvaluadorPromociones`, de `PoliticaTarifa` y de `Reloj`.
GRASP Experto	`Viaje` calcula duración y distancia, `Estacion` su ocupación y alertas, `CuentaUsuario` su saldo.
GRASP Creador	`ControladorViajes` crea el `Viaje`, `CuentaUsuario` crea sus `MovimientoCuenta` y `Operador.solicitarOrden` crea la `OrdenMantenimiento`.
GRASP Controlador	La interfaz de consola solo conoce a los controladores y no contiene reglas de negocio.
GRASP Polimorfismo	La jerarquía `Vehiculo`, `BicicletaComun` y `VehiculoElectrico` resuelve batería y estado tras el viaje sin condicionales por tipo.

4. Dificultades encontradas

4.1 Dificultades técnicas

- **Tiempo y límites temporales.** Las reglas de 2 minutos, 10 minutos, 120 minutos, 24 horas y 30 días no se pueden probar con el reloj real. Se resolvió con la interfaz `Reloj`, inyectada en controladores y servicios, y con `RelojPrueba`, que permite avanzar el tiempo en las pruebas.
- **Combinar recargo, bonificación y gracia.** El precio depende de los minutos bonificados, del desbloqueo gratis y del recargo del 50 % pasados los 120 minutos, además de los minutos descontados por estación llena. `TarifaBase.calcularPrecio` calcula por separado los minutos normales y los que llevan recargo, y los tests de límites (120 y 130 minutos) validan el resultado.
- **Orden de las operaciones al devolver.** El primer viaje gratis depende de que el usuario no tenga viajes finalizados, y el plan consume un viaje al finalizar. Por eso se evalúan las promociones antes de llamar a `registrarViajeFinalizado`.
- **Estación llena.** Solo cuenta el primer intento fallido, el viaje sigue en curso después de lanzar la excepción y el instante debe persistirse para no perder la gracia al reiniciar.

- **Mantenimiento obligatorio con el vehículo en uso.** No se puede mandar a reparación un vehículo que está en viaje. Se resolvió con el atributo `mantenimientoPendiente`, que se aplica al cerrar el viaje.
- **Persistencia de un grafo de objetos.** Los archivos CSV se cargan en orden de dependencias (personal, estaciones, vehículos, usuarios, viajes, órdenes, traslados y notas). Los vehículos en uso no están en ninguna estación. Hubo que reconstruir el estado sin disparar reglas de negocio, no volver a escribir los archivos durante la carga y restaurar los contadores de identificadores. Además, la escritura se hace primero en un archivo temporal que luego reemplaza al original, para no dejarlo a medias, y los caracteres especiales se escapan para no romper el formato.
- **Importes monetarios.** Se usó `BigDecimal` con 2 decimales y redondeo `HALF_UP` (clase `Dinero`) para evitar errores de coma flotante.
- **Mantener coherentes diagramas y código.** Al codificar aparecieron clases e interfaces que no estaban en el diagrama final (sección 3.1).]

4.2 Dificultades de organización del equipo

Se encontró mucha dificultad en comprender el uso de gitHub y la coordinación entre los participantes del equipo

5. Pruebas

Pruebas unitarias (JUnit 5). Se escribieron sobre un `Escenario` común: un sistema completo en memoria con un reloj controlable.

Clase de prueba	Qué verifica
`TarifasYPromocionesTest`	Cálculo de precio, recargo del 50 %, saldo mínimo, selección de tarifa vigente, primer viaje gratis, plan mensual y elección de la promoción más favorable.
`ControladorViajesTest`	Retiro, devolución y cancelación, incluidas las ramas de error de los diagramas de secuencia (inhabilitado, viaje abierto, saldo, vehículo no disponible, estación llena y gracia de 10 minutos).
`ControladorUsuariosTest`	Registro, medios de pago, recargas, rehabilitación, plan mensual e historial.

`MantenimientoOperacionesReportesTest`	Fallas, ciclo completo de una orden, redistribución, notas de ajuste y reportes.
`ServicioControlViajesTest`	Vehículo perdido a las 24 horas, cargo único e inhabilitación.
`EstacionViajeCuentaTest`	Ocupación y alertas, duración, distancia, gracia, orden natural de viajes y movimientos de cuenta.
`VehiculoTest`	Batería, estados, mantenimiento obligatorio y polimorfismo.
`RepositorioArchivosTest`	Ida y vuelta completa de la persistencia, archivos dañados y escape de caracteres especiales.

Pruebas funcionales. El plan con entradas, resultado esperado y resultado obtenido está en `docs/PLAN_DE_PRUEBAS_FUNCIONALES.md`. Los límites temporales se cubren solo con pruebas unitarias, porque la consola usa el reloj real

6. Distribución de tareas

El aporte de cada integrante puede verificarse en el historial del repositorio (`git shortlog -sn --all`) y en los pull requests.

Integrante	Tareas realizadas
Astudillo Solis, Valentina	Diagrama de clases inicial, final y UML. Estructurado básico del código para la organización y armado posterior. Elaboración y perfeccionamiento de las Excepciones. Armado de la carpeta de Dominio, junto a sus

	sub-carpetas Personal, Pagos, Promociones, Tarifas y Vehículos. Corrección de diagrama de clase inicial según las indicaciones de la profesora
Pacheco, Maitena Alumine	Armado y estructuración de la carpeta de Dominio, junto a la elaboración de sus clases y relaciones. Presentó organización y codificación de las sub-carpetas, junto a perfeccionamiento de sus clases. Junto al README
Romero Chacon, Marcos Ignacio	Correcciones generales del código, junto a corrección de la estructura, codificación y esquemas. Codificación general en las carpetas de Persistencia y Soporte. Desarrollador y organizador del Informe Final, a su vez realizó la documentación específica y el README.

Sugerencia de áreas para ordenar el detalle: análisis y casos de uso; diagramas UML; patrones SOLID y GRASP; dominio y vehículos; tarifas y promociones; controladores y servicios; persistencia; interfaz de consola; pruebas unitarias; plan de pruebas funcionales; README e informes.

7. Reflexión del grupo

Qué aprendimos.

- Pensar el diseño antes de programar ahorró trabajo. Tener los casos de uso y los diagramas de secuencia permitió programar los controladores siguiendo el orden de validaciones ya definido.
- Los principios de diseño tienen un efecto concreto. Gracias a DIP y a la inyección del reloj se pudieron probar reglas que dependen del tiempo, y gracias a OCP y al polimorfismo agregar un tipo de vehículo o de promoción no obliga a tocar el flujo principal.

- Documentar los supuestos (S01 a S25) evitó discusiones durante la codificación, porque cada regla ambigua ya tenía una decisión escrita.
- Persistir objetos relacionados es más complejo que guardar texto: el orden de carga, las referencias entre objetos y el estado de los vehículos en uso exigen cuidado.

Qué mejoraríamos con más tiempo.

- Agregar el menú del administrador para gestionar tarifas, promociones y el precio del plan (CU-23 y CU-24), y persistir esa configuración.
- Hacer que `Sistema` dependa de las interfaces de repositorio y no de `RepositorioMemoria`, y reducir los métodos `restaurar*` del dominio con un mecanismo de carga más aislado.
- Definir `equals` en los medios de pago, hoy comparados por identidad al validar una recarga.
- Mejorar el manejo de errores inesperados en la consola y las alertas, que muestran la última generada aunque sea de una operación anterior.
- Proteger mejor los datos sensibles: el número de tarjeta se guarda completo en `medios_pago.csv`, algo aceptable en un trabajo práctico pero que en un sistema real se resolvería con tokenización.
- Una interfaz gráfica

8. Conclusión

El sistema cumple los casos de uso mínimos de la Propuesta C, incluidos los dos modelados en secuencia, y respeta los requisitos técnicos de la consigna: herencia y polimorfismo, encapsulamiento, colecciones con `Comparable` y `Comparator`, excepciones propias, persistencia en archivos, interfaz de consola con menús y pruebas unitarias. Los apartamientos del diseño original están documentados y justificados en la sección 3.